

Taller Práctico Avanzado: Securización Profesional de APIs con Spring Security, JWT y Roles

Tomando como punto de partida el **Sistema de Gestión de Proyectos** (que cuenta con las entidades `Proyecto`, `Empleado` y `Tarea` en el paquete `com.krakedev.proyectos` y se conecta a la base de datos `proyectos_db`), integrarás **Spring Security**, tokens **JWT (JSON Web Tokens)**, cifrado unidireccional con **BCrypt**, control de sesiones a través de una **Blacklist (Lista Negra) en memoria** y autorización granular basada en roles.



Hito 1: Capa de Acceso de Usuarios y Hasheo con BCrypt

En este primer hito, sentaremos las bases del control de acceso persistiendo la información de nuestros usuarios y asegurando que las contraseñas nunca se almacenen en texto plano.

IMPORTANT

Integración Continua y Segura desde el Inicio En este taller adoptamos una mentalidad de seguridad por diseño (*Security by Design*). Queda estrictamente prohibido iniciar con contraseñas en texto plano o realizar validaciones manuales de credenciales con métodos como `.equals()`. Desde el primer instante, implementaremos un esquema de hasheo seguro y unidireccional utilizando **BCrypt** en el servicio de registro (`BCrypt.hashpw`) y en el método de login/autenticación (`BCrypt.checkpw`).



1.1 Estructura Física en PostgreSQL

Crea la tabla `usuarios` dentro de tu base de datos `proyectos_db`. La especificación formal de la tabla se detalla en el siguiente esquema:

```
CREATE TABLE usuarios (  
  id SERIAL PRIMARY KEY,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  password VARCHAR(100) NOT NULL,  
  rol VARCHAR(20) NOT NULL  
);
```



1.2 Configuración del pom.xml (Criptografía)

Instrucción: Abre el archivo `pom.xml` de tu proyecto anterior e incorpora las dependencias necesarias. Localiza la sección `<dependencies>` y añade la dependencia de `bcrypt` para habilitar el algoritmo de hasheo unidireccional seguro:

```
xml  
<dependency>
```

```
<groupId>org.mindrot</groupId>
<artifactId>jbcrypt</artifactId>
<version>0.4</version>
</dependency>
```

⚙️ 1.3 Verificación de Conexión (`application.properties`)

Instrucción: Abre el archivo `src/main/resources/application.properties` de tu proyecto anterior y asegúrate de mantener tu conexión activa a `proyectos_db`. Confirma que las credenciales de PostgreSQL y la URL sean correctas para que la aplicación persista los usuarios de manera adecuada:

`properties`

```
spring.datasource.url=jdbc:postgresql://localhost:5432/proyectos_db
spring.datasource.username=tu_usuario_postgres
spring.datasource.password=tu_contrasena_postgres
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

☕ 1.4 Entidad de Dominio `Usuario.java`

Instrucción: En tu proyecto anterior, crea la clase `Usuario.java` en el paquete de entidades existente `com.krakedev.proyectos.entidades`. Esta clase debe estar mapeada mediante JPA a la tabla física `usuarios`.

🏛️ 1.5 Repositorio y Servicio de Autenticación

Crea e implementa las siguientes clases para gestionar el acceso a los datos de usuario y procesar la lógica de negocio asociada.

1. Interfaz `UsuarioRepository.java`

Instrucción: Créala en el paquete de repositorios existente

`com.krakedev.proyectos.repositories:`

`java`

```
package com.krakedev.proyectos.repositories;
import com.krakedev.proyectos.entidades.Usuario;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;
public interface UsuarioRepository extends JpaRepository<Usuario,
Integer> {
    Optional<Usuario> findByUsername(String username);
}
```

2. Clase `UsuarioService.java`

Instrucción: Créala en el paquete de servicios existente `com.krakedev.proyectos.services`. Esta clase contiene la lógica de encriptación unidireccional de contraseñas mediante BCrypt desde el primer instante, asegurando que nunca se compare texto plano directamente con `.equals()`

1.6 Controlador de Autenticación Temporal

Instrucción: Crea la clase `AuthController.java` en el paquete de controladores existente `com.krakedev.proyectos.controllers` para exponer los flujos iniciales de entrada y registro:

```
}
```

IMPORTANT

 **PUNTO DE CONTROL 1 (WhatsApp):** Realiza el registro de al menos dos usuarios (uno con rol `ADMIN` y otro con `USER`) mediante Postman en el endpoint `/api/auth/registrar`. Toma una captura de pantalla de tu cliente de PostgreSQL (pgAdmin o similar) donde se vea claramente el resultado de realizar un `SELECT * FROM usuarios;`. Las contraseñas deben visualizarse codificadas con la estructura característica de BCrypt (empezando con `$2a$10$...`).

Hito 2: Generación del Token JWT y Logout con Blacklist

En este hito, evolucionaremos nuestra API de sesión con estado a una arquitectura **Stateless** (sin estado) empleando tokens firmados digitalmente. También resolveremos el reto de invalidación inmediata del token al cerrar la sesión.

2.1 Configuración de Dependencia JWT

```
<dependency>
  <groupId>com.auth0</groupId>
  <artifactId>java-jwt</artifactId>
  <version>4.4.0</version>
</dependency>
```

2.2 Diseñando el Motor de Firmado (`JwtUtil.java`)

Instrucción: Crea la clase `JwtUtil.java` dentro de un nuevo paquete de seguridad llamado `com.krakedev.proyectos.security`. Esta clase encapsulará el secreto y las operaciones de creación y verificación del token digital.

2.3 Implementación de Invalidación: Lista Negra en Memoria

Dado que un token JWT es auto-contenido y sin estado, seguirá siendo matemáticamente válido en el cliente hasta que llegue su tiempo de expiración. Para poder realizar un "Logout" real en el servidor, implementaremos una **Lista Negra (Blacklist)** de tokens en memoria concurrente.

1. Clase `TokenBlacklistService.java`

Instrucción: Créala en el paquete de servicios existente `com.krakedev.proyectos.services` utilizando una estructura segura para acceso concurrente en entornos multi-hilo:

```
}
```

2.4 Actualización de Controladores

Instrucción: Abre el archivo `AuthController.java` ubicado en `com.krakedev.proyectos.controllers` que creaste en el Hito 1. Modifícalo para inyectar la invalidación del token Actualización del Login y Endpoint `/api/auth/logout`:
Reemplaza e integra en `AuthController.java`:

TIP

Consistencia del Formato del Token Asegúrate de que las peticiones al endpoint `/logout` envíen el token en el formato estandarizado `Bearer <TOKEN_JWT>`, garantizando que el método `authHeader.substring(7)` funcione correctamente sin generar fallos lógicos de índice de cadena de caracteres.

Hito 3: Integración de Spring Security y Filtro Personalizado

En este hito, daremos el paso decisivo: instalaremos **Spring Security** para que actúe como escudo central de nuestra API y delegaremos la validación de peticiones a un filtro que se ejecute una vez por petición.

3.1 Integración de Dependencia Spring Security

Instrucción: Abre de nuevo el archivo `pom.xml` de tu proyecto e incorpora el iniciador oficial de seguridad de Spring Boot para activar la protección de las rutas. Añade la siguiente dependencia dentro de `<dependencies>`:

xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

3.2 Guardián de Peticiones (`JwtAuthenticationFilter.java`)

Instrucción: Crea la clase `JwtAuthenticationFilter.java` en el paquete de seguridad `com.krakedev.proyectos.security`. Esta clase interceptará cada solicitud entrante, comprobará que no esté en la lista negra, validará la firma de seguridad y configurará el contexto de seguridad oficial de Spring:

⚙️ 3.3 Configuración Central de Seguridad (`SecurityConfig.java`)

Instrucción: Crea la clase `SecurityConfig.java` dentro del paquete de seguridad `com.krakedev.proyectos.security`. Aquí definirás qué rutas son públicas y cuáles requerirán validación del filtro JWT.

🏛️ 3.4 Refactorización del Perfil de Usuario

Instrucción: Abre el archivo `AuthController.java` en `com.krakedev.proyectos.controllers` y agrega la refactorización para el endpoint `/api/auth/perfil` de modo que lea la identidad directamente del flujo seguro de Spring Security en lugar de procesar manualmente la cabecera HTTP:

🔧 Hito 4: Control de Accesos por Roles

En este hito, configuraremos las políticas de negocio granulares aplicando roles específicos en nuestros controladores relacionales de negocio y manejando con elegancia los bloqueos de acceso no autorizados.

📋 4.1 Securitización con `@PreAuthorize` en Controladores de Negocio

Instrucción: Abre tus controladores de negocio del proyecto anterior (ubicados en `com.krakedev.proyectos.controllers`) para aplicar control de acceso declarativo basado en roles. Deberás guiarte por las siguientes directivas de modificación exacta para integrar la seguridad:

1. **Abre el archivo `ProyectoController.java`:** Ubica el método `crearProyecto` y agrégale en la parte superior la anotación `@PreAuthorize("hasRole('ADMIN')")`. Ubica el método `eliminarProyecto` y agrégale también la anotación `@PreAuthorize("hasRole('ADMIN')")`. Finalmente, ubica el método `listarProyectos` y agrégale la anotación `@PreAuthorize("hasAnyRole('ADMIN','USER')")`.
 2. **Abre el archivo `EmpleadoController.java`:** Ubica el método `listarEmpleados` y agrégale en la parte superior la anotación `@PreAuthorize("hasAnyRole('ADMIN','USER')")`.
 3. **Abre el archivo `TareaController.java`:** Ubica el método `crearTarea` y agrégale la anotación `@PreAuthorize("hasRole('ADMIN')")`. Ubica también el método `listarTareas` y agrégale la anotación `@PreAuthorize("hasAnyRole('ADMIN','USER')")`.
-

IMPORTANT

 **PUNTO DE CONTROL 2 (WhatsApp): Instrucción:** Ajusta las pruebas de Postman creadas en el taller anterior para que cada solicitud incluya la cabecera `Authorization` con el token JWT obtenido del login.

1. Inicia sesión en Postman con las credenciales de un desarrollador (rol `USER`) en `/api/auth/login`. Copia su token JWT.
 2. Diseña una petición `POST` al endpoint `http://localhost:8080/api/proyectos` para registrar un proyecto nuevo, adjuntando en la cabecera `Authorization` el token con el prefijo `Bearer` (ej. `Bearer <TOKEN_JWT_DE_USER>`).
 3. Ejecuta la petición. Toma una captura de pantalla completa donde se observe el código de estado **403 Forbidden** de la respuesta
-

¡Mucho éxito en la securización de tu backend! Pon a prueba cada uno de tus endpoints y valida la consistencia de cada token en tu camino a ser un desarrollador backend profesional. 🚀



Entrega Final

- Envía el **enlace de tu repositorio de GitHub**
- **Capturas de pantalla** solicitadas a través de WhatsApp.